

1. Linked Lists – Create a Swap Method

```
def swapCurrentNext(self):
    if self.Header is None or self.Current is None or self.Current.Next is None:
        return -1

    nextNode = self.Current.Next

    prev = None
    if self.Current != self.Header:
        prev = self.Header
        while prev.Next != self.Current:
            prev = prev.Next

    self.Current.Next = nextNode.Next
    nextNode.Next = self.Current

    if prev is None:
        self.Header = nextNode
    else:
        prev.Next = nextNode

    self.Current = nextNode
    return 0
```

2. Asymptotic Notations – Computing the Complexity

(A) What is the complexity of A?

$O(n + n) = O(2n) = O(n)$, cause of the 2 loops.

(B) What is the complexity of B?

$O(n * n) = O(n^2)$, cause of the 2 nested loops.

(C) What is the complexity of C?

$O(n * n^2) = O(n^3)$, since each $O(n)$ call has a complexity of $O(n^2)$.

(D) What is the complexity of D?

$O(n^2 * \log n)$, since each $O(n^2)$ calls $O(\log n)$.

3. Brute-Force Algorithm – Create the Difference of Two Sets

(A) Brute force function

```
def bruteForce(A, B):  
    C = []  
    for a in A:  
        value = False  
        for b in B:  
            if a == b:  
                value = True  
                break  
        if not value:  
            C.append(a)  
    return C  
  
A = [2, 4, 6]  
B = [3, 4, 5]  
result = bruteForce(A, B)  
print("Difference of sets A and B:", result)
```

(B) Trace the algorithm

```
A = [20, 40, 70, 30, 10, 80, 50, 90, 60]  
B = [35, 45, 55, 60, 50, 40]  
C = []
```

```
20 is not in B, add to C  
[20]  
40 is in B, do not add to C  
70 is not in B, add to C  
[20, 70]  
30 is not in B, add to C  
[20, 70, 30]  
10 is not in B, add to C  
[20, 70, 30, 10]  
80 is not in B, add to C  
[20, 70, 30, 10, 80]  
50 is in B, do not add to C  
90 is not in B, add to C  
[20, 70, 30, 10, 80, 90]  
60 is in B, do not add to C
```

Diff of sets A – B is [20, 70, 30, 10, 80, 90]

(C) Asymptotic analysis

The maximum number of comparisons is $m * n$. So, the Big O is $O(m * n)$.

(4) Recursion – Breadth First Search and Depth First Search

(A) Adjacency lists

A: B, D

B: A, C, G

C: A, B

D: E, F

E: F

F: B

G: C, F

(B) BFS Algorithm – Create the BFS Traversal of a Graph

A:0, B:1, C:2, D:3, E:4, F:5, G:6

A = [B, D]

Vertex A is visited

Current array: [0, -1, -1, -1, -1, -1, -1]

Vertex B and D enqueued, Current queue: [B, D]

Vertex A is dequeued, Current queue: [B, D]

B = [A, C, G]

Vertex B is visited

Current array: [0, 1, -1, -1, -1, -1, -1]

Vertex A is already visited

Vertex C and G enqueued, Current queue: [D, C, G]

Vertex B is dequeued, Current queue: [D, C, G]

D = [E, F]

Vertex D is visited

Current array: [0, 1, -1, 3, -1, -1, -1]

Vertex E and F enqueued, Current queue: [C, G, E, F]

Vertex D is dequeued, Current queue: [C, G, E, F]

C = [A, B]

Vertex C is visited

Current array: [0, 1, 2, 3, -1, -1, -1]

Vertex A is already visited

Vertex B is already visited

Vertex C is dequeued, Current queue: [G, E, F]

G = [C, F]

Vertex G is visited

Current array: [0, 1, 2, 3, -1, -1, 6]

Vertex C is already visited

Vertex F is enqueued, Current queue: [E, F]

Vertex G is dequeued, Current queue: [E, F]

E = [F]

Vertex E is visited

Current array: [0, 1, 2, 3, 4, 5, 6]

Vertex F is enqueued, Current queue: [F]

Vertex E is dequeued, Current queue: [F]

F = [B]
Vertex F is visited
Current array: [0, 1, 2, 3, 4, 5, 6]
Vertex B is already visited
Vertex F is dequeued, Current queue: []

Final Array: [A:0, B:1, D:2, C:3, G:4, E:5, F:6]

(C) DFS Algorithm – Create the DFS Traversal of a Graph

A:0, B:1, C:2, D:3, E:4, F:5, G:6

A = [B, D]
Vertex A is visited
Current array: [0, -1, -1, -1, -1, -1, -1]
Visit Vertex B

B = [A, C, G]
Vertex B is visited
Current array: [0, 1, -1, -1, -1, -1, -1]
Visit Vertex C

C = [A, B]
Vertex C is visited
Current array: [0, 1, 2, -1, -1, -1, -1]
Vertex A and B are already visited
Go to Vertex G

G = [C, F]
Vertex G is visited
Current array: [0, 1, 2, -1, -1, -1, 6]
Vertex C is already visited
Go to Vertex F

F = [B]
Vertex F is visited
Vertex B is already visited
Current array: [0, 1, 2, -1, -1, 5, 6]
Go to Vertex D

D = [E, F]
Vertex D is visited
Current array: [0, 1, 2, 3, -1, -1, -1]
Go to Vertex E

E = [F]
Vertex E is visited
Current array: [0, 1, 2, 3, 4, -1, -1]
Vertex F is already visited

Final Array: [A:0, B:1, C:2, G:3, F:4, D:5, E:6]

5. Recursion – Master Method

(A) $T(n) = 4T(n/2) + n^3$

$$a = 4$$

$$b = 2$$

$$f(n) = n^3$$

$$\log_b(a) = \log(4)/\log(2) = 2$$

Worst case: $T(n) = \Theta(n^3)$

(B) $T(n) = 4T(n/2) + n^2$

$$a = 4$$

$$b = 2$$

$$f(n) = n^2$$

$$\log_b(a) = \log(4)/\log(2) = 2$$

Worst case: $T(n) = \Theta(n^2)$

(C) $T(n) = 4T(n/2) + n$

$$a = 4$$

$$b = 2$$

$$f(n) = n$$

$$\log_b(a) = \log(4)/\log(2) = 2$$

$$f(n) = n^{\log_b(a)} = n^2$$

Worst case: $T(n) = \Theta(n^2)$

(6) Decrease-and-Conquer Algorithm – Maximum Element in Array

(A) Recursive decrease-and-conquer algorithm

```
def Maximum(A, right):  
    if right == 0:  
        return A[0]  
    else:  
        max = Maximum(A, right - 1)  
        return max if max > A[right] else A[right]
```

```
A = [5, 13, 9, 10]  
print(Maximum(A, len(A) - 1))
```

(B) Trace the algorithm

```
def Maximum(A, right):  
    if right == 0:  
        print(f"Base case: return A[0] = {A[0]}")  
        return A[0]  
    else:  
        max = Maximum(A, right - 1)  
        current = A[right]  
        max_val = max if max > current else current
```

```

        print(f"Comparing A[{right}] = {current} with max = {max} -> max = {max_val}")
    return max_val

```

```

A = [17, 62, 49, 73, 26, 51]
print("Maximum element is:", Maximum(A, len(A) - 1))

```

```

Base case: return A[0] = 17
Comparing A[1] = 62 with max = 17 -> max = 62
Comparing A[2] = 49 with max = 62 -> max = 62
Comparing A[3] = 73 with max = 62 -> max = 73
Comparing A[4] = 26 with max = 73 -> max = 73
Comparing A[5] = 51 with max = 73 -> max = 73
Maximum element is: 73

```

(C) Recursive relation

```

T(n) = T(n - 1) + 1 , T(1) = 0
T(n) = T(n - 1) + 1
T(n) = T(n - 2) + 2
T(n) = T(n - 3) + 3

```

Big O is $O(n)$

(7) Divide-and-Conquer Algorithms – Mergesort and Quicksort

(A) Worst case for Mergesort and Quicksort

```

Mergesort worst case is  $O(n \log n)$ 
Quicksort worst case is  $O(n^2)$ 

```

(B) Average case for Mergesort and Quicksort

```

Mergesort average case is  $O(n \log n)$ 
Quicksort average case is  $O(n \log n)$ 

```

(C) Trace the Mergesort algorithm

```

A = [127, 48, 62, 51, 198, 17, 52, 209]

```

```

Left: [127] Right: [48]
Merged: [48, 127]

```

```

Left: [62], Right: [51]
Merged: [51, 62]

```

```

Left: [198], Right: [17]
Merged: [17, 198]

```

```

Left: [52], Right: [209]
Merged: [52, 209]

```

```

Left: [48, 127], Right: [51, 62]
Merged: [48, 51, 62, 127]

```

```

Left: [17, 198], Right: [52, 209]

```

Merged: [17, 52, 198, 209]

Sorted array: [17, 48, 51, 52, 62, 127, 198, 209]

(D) Trace the Quick algorithm

```
def quickSort(A, left, right, swaps, recursiveCalls):
    if left < right:
        recursiveCalls += 1
        print(f"\nQuickSort called on A[{left}:{right + 1}] -> {A[left:right+1]}")
        pivotIndex, swaps = lumuto(A, left, right, swaps)
        print(f"Pivot placed at index {pivotIndex}, value = {A[pivotIndex]}")
        print(f"Array after partition: {A}")
        swaps, recursiveCalls = quickSort(A, left, pivotIndex - 1, swaps, recursiveCalls)
        swaps, recursiveCalls = quickSort(A, pivotIndex + 1, right, swaps, recursiveCalls)
    return swaps, recursiveCalls
```

```
def lumuto(A, left, right, swaps):
    pivot = A[right]
    print(f"pivot A[{right}] = {pivot}")
    i = left - 1
    for j in range(left, right):
        print(f"Comparing A[{j}] = {A[j]} with pivot {pivot}")
        if A[j] > pivot:
            i += 1
            A[i], A[j] = A[j], A[i]
            swaps += 1
        print(f"Swapped A[{i}] and A[{j}] -> {A}")
    A[i+1], A[right] = A[right], A[i+1]
    swaps += 1
    print(f" Swapped pivot A[{i+1}] and A[{right}] -> {A}")
    return i + 1, swaps
```

A = [127, 48, 62, 51, 198, 17, 52, 209]

```
print("Original array:", A)
swaps, recursiveCalls = quickSort(A, 0, len(A) - 1, 0, 0)
print("Sorted array:", A)
```

Original array: [127, 48, 62, 51, 198, 17, 52, 209]

QuickSort called on A[0:8] -> [127, 48, 62, 51, 198, 17, 52, 209]

pivot A[7] = 209

Comparing A[0] = 127 with pivot 209

Comparing A[1] = 48 with pivot 209

Comparing A[2] = 62 with pivot 209

Comparing A[3] = 51 with pivot 209

Comparing A[4] = 198 with pivot 209

Comparing A[5] = 17 with pivot 209

Comparing A[6] = 52 with pivot 209

Swapped pivot A[0] and A[7] -> [209, 48, 62, 51, 198, 17, 52, 127]

Pivot placed at index 0, value = 209

Array after partition: [209, 48, 62, 51, 198, 17, 52, 127]

QuickSort called on A[1:8] -> [48, 62, 51, 198, 17, 52, 127]
pivot A[7] = 127
Comparing A[1] = 48 with pivot 127
Comparing A[2] = 62 with pivot 127
Comparing A[3] = 51 with pivot 127
Comparing A[4] = 198 with pivot 127
Swapped A[1] and A[4] -> [209, 198, 62, 51, 48, 17, 52, 127]
Comparing A[5] = 17 with pivot 127
Comparing A[6] = 52 with pivot 127
Swapped pivot A[2] and A[7] -> [209, 198, 127, 51, 48, 17, 52, 62]
Pivot placed at index 2, value = 127
Array after partition: [209, 198, 127, 51, 48, 17, 52, 62]

QuickSort called on A[3:8] -> [51, 48, 17, 52, 62]
pivot A[7] = 62
Comparing A[3] = 51 with pivot 62
Comparing A[4] = 48 with pivot 62
Comparing A[5] = 17 with pivot 62
Comparing A[6] = 52 with pivot 62
Swapped pivot A[3] and A[7] -> [209, 198, 127, 62, 48, 17, 52, 51]
Pivot placed at index 3, value = 62
Array after partition: [209, 198, 127, 62, 48, 17, 52, 51]

QuickSort called on A[4:8] -> [48, 17, 52, 51]
pivot A[7] = 51
Comparing A[4] = 48 with pivot 51
Comparing A[5] = 17 with pivot 51
Comparing A[6] = 52 with pivot 51
Swapped A[4] and A[6] -> [209, 198, 127, 62, 52, 17, 48, 51]
Swapped pivot A[5] and A[7] -> [209, 198, 127, 62, 52, 51, 48, 17]
Pivot placed at index 5, value = 51
Array after partition: [209, 198, 127, 62, 52, 51, 48, 17]

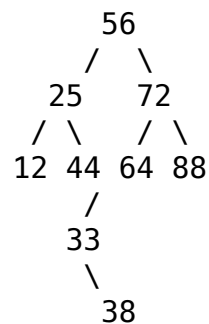
QuickSort called on A[6:8] -> [48, 17]
pivot A[7] = 17
Comparing A[6] = 48 with pivot 17
Swapped A[6] and A[6] -> [209, 198, 127, 62, 52, 51, 48, 17]
Swapped pivot A[7] and A[7] -> [209, 198, 127, 62, 52, 51, 48, 17]
Pivot placed at index 7, value = 17
Array after partition: [209, 198, 127, 62, 52, 51, 48, 17]

Sorted array: [209, 198, 127, 62, 52, 51, 48, 17]

(8) Transform-and-Conquer Algorithms – A VL Trees

(A) Show tree after adding 38 and not balancing

We attach it to **33** cause its less than **56** go to the left
Its greater than **25** go right
Its less than **44** go left
Its greater than **33** go right



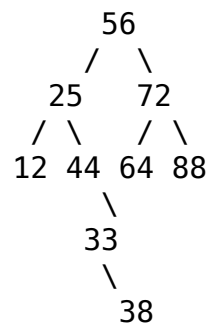
(B) Describe what possible rotations

Right rotation on 44

Right rotation on 33

(C)

Right rotation on 44



Right rotation on 33

